PANDAS

DATAFRAMES



Series

	apples
0	3
1	2
2	0
3	1

+

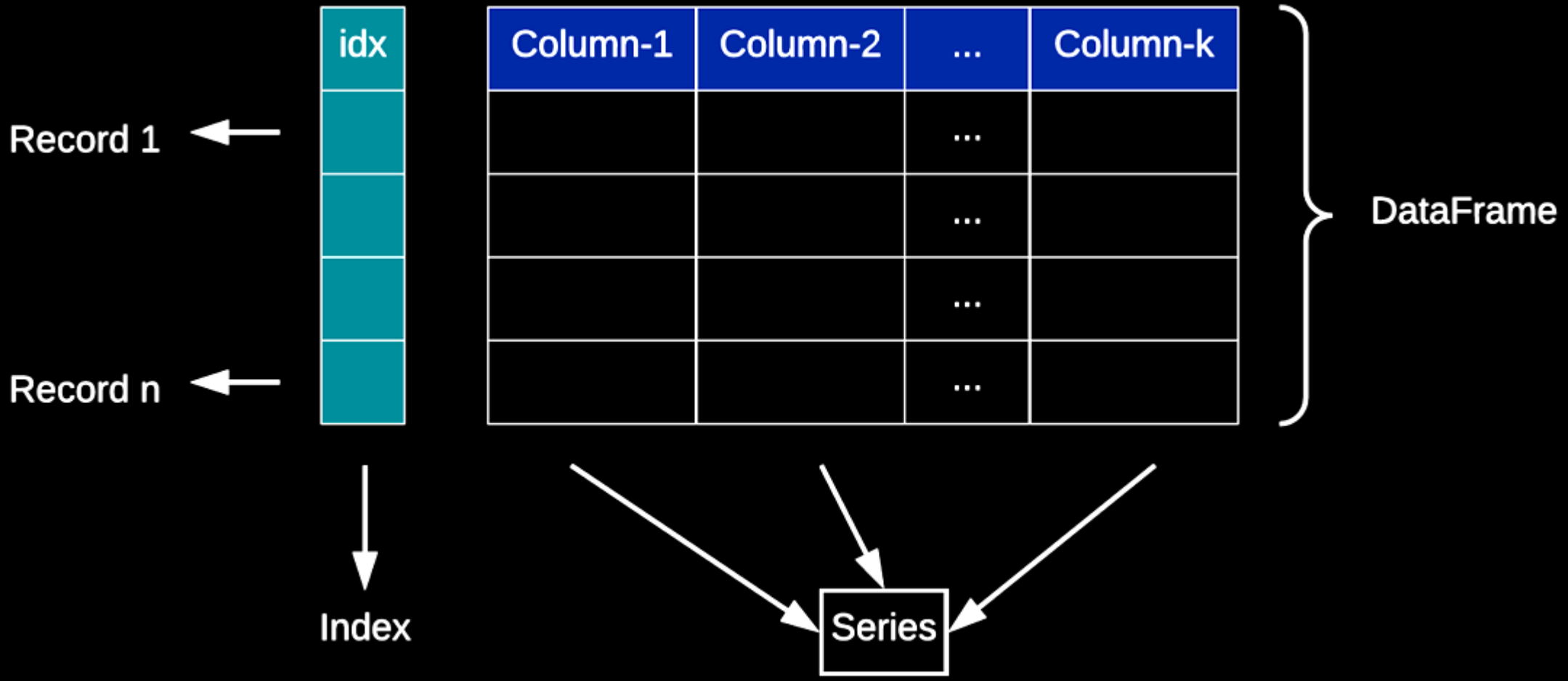
Series

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2


idx
Column-1
Column-2
...
Column-k
Record 1
Record n
DataFrame
Index
Series

Series

Definition: A Series is a one-dimensional array-like object containing a sequence of values and an associated array of data labels, called its index.

Structure: Think of a Series as a column in an Excel sheet. Each value has a unique label, which acts like the row name.

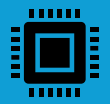
Creation: A Series can be created from a list, dictionary, or even another Series.



Indexed Data: Each element has a unique label (index) which makes data manipulation straightforward and efficient.



Versatility: It can handle different types of data—numeric, string, datetime, etc.—in one structure.



Data Alignment: It aligns data by its labels automatically during operations, which is handy for joining data sets and performing arithmetic operations.



Methods and Functions: It comes with a variety of built-in methods for data cleaning, transformation, and computation. For example, you can easily calculate the mean, sum, or count of a Series.



Time Series Data: If you're working with time series data, a Pandas Series provides powerful time-based functionality.

Series Example

```
import pandas as pd

# Creating a Pandas Series
data = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

# Accessing data by label
print(data['b'])  # Output: 20

# Mathematical operations
print(data.mean())  # Output: 25.0
```

Dataframe



Definition: A DataFrame is a two-dimensional, size-mutable, and potentially heterogeneous tabular data structure with labeled axes (rows and columns).



Structure: Imagine a DataFrame as an Excel sheet with rows and columns. Each column in a DataFrame is a Series.



Creation: DataFrames can be created from dictionaries, lists, or other DataFrames.



Data Organization: A DataFrame organizes data in a tabular format, similar to a spreadsheet or SQL table, making it intuitive and accessible.



Data Manipulation: It provides powerful tools to clean, filter, aggregate, and transform data efficiently.



Data Analysis: With built-in methods for statistical operations, a DataFrame facilitates in-depth data analysis.

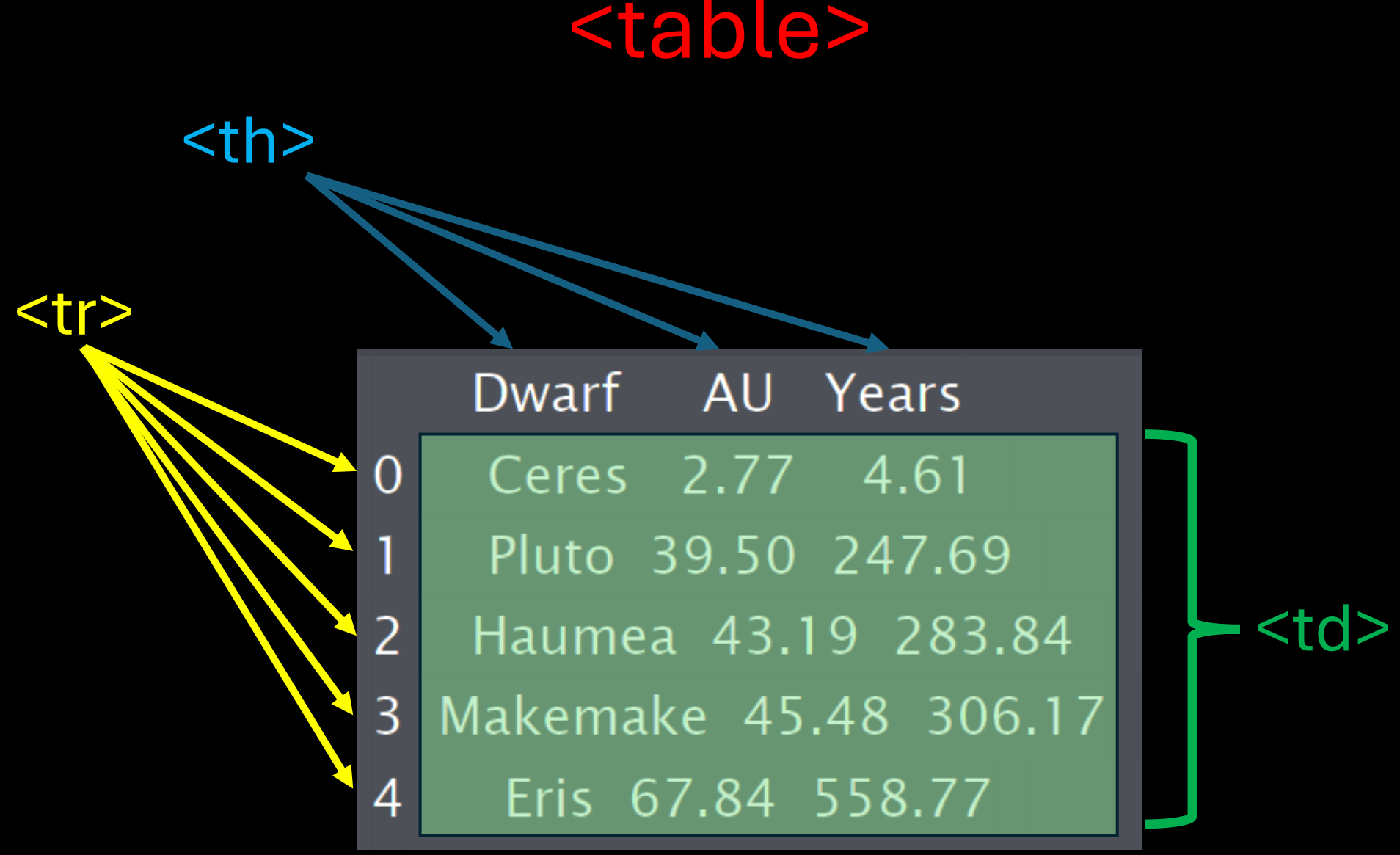


Interoperability: DataFrames can easily interface with other data sources like CSV files, databases, and Excel sheets.

Datafame Example

```
import pandas as pd

# dictionary
pdict = {
'Dwarf': {0: 'Ceres', 1: 'Pluto', 2: 'Haumea', 3: 'Makemake', 4: 'Eris'},
'AU': {0: 2.77, 1: 39.5, 2: 43.19, 3: 45.48, 4: 67.84},
'Years': {0: 4.61, 1: 247.69, 2: 283.84, 3: 306.17, 4: 558.77}
}
df = pd.DataFrame(pdict)
print(df)
print(f'dwarf: {df.iloc[1,0]}, AU: {df.iloc[1,1]}, Years: {df.iloc[1,2]}')
```


<table>
<th>
<tr>
Dwarf AU Years
0 Ceres 2.77 4.61
1 Pluto 39.50 247.69
2 Haumea 43.19 283.84
3 Makemake 45.48 306.17
4 Eris 67.84 558.77
<td>

```
import pandas as pd
dfcsv = pd.read_csv("Dwarfs.csv",delimiter=',')
print(dfcsv)
```

Dwarfs.csv

```
Dwarf,AU,Years
Ceres,2.77,4.61
Pluto,39.5,247.69
Haumea,43.19,283.84
Makemake,45.48,306.17
Eris,67.84,558.77
```

Dwarfs DataFrame

```
     Dwarf     AU   Years
0    Ceres   2.77    4.61
1    Pluto  39.50  247.69
2   Haumea  43.19  283.84
3 Makemake  45.48  306.17
4     Eris  67.84  558.77
```

```
import pandas as pd
dframe = pd.read_csv("Dwarfs.csv",delimiter=',')
dframe.to_excel('Dwarfs.xlsx',index=False)
```

Dwarfs DataFrame

	Dwarf	AU	Years
0	Ceres	2.77	4.61
1	Pluto	39.50	247.69
2	Haumea	43.19	283.84
3	Makemake	45.48	306.17
4	Eris	67.84	558.77

Dwarfs Excel

	A	B	C
1	**Dwarf**	**AU**	**Years**
2	Ceres	2.77	4.61
3	Pluto	39.5	247.69
4	Haumea	43.19	283.84
5	Makemake	45.48	306.17
6	Eris	67.84	558.77